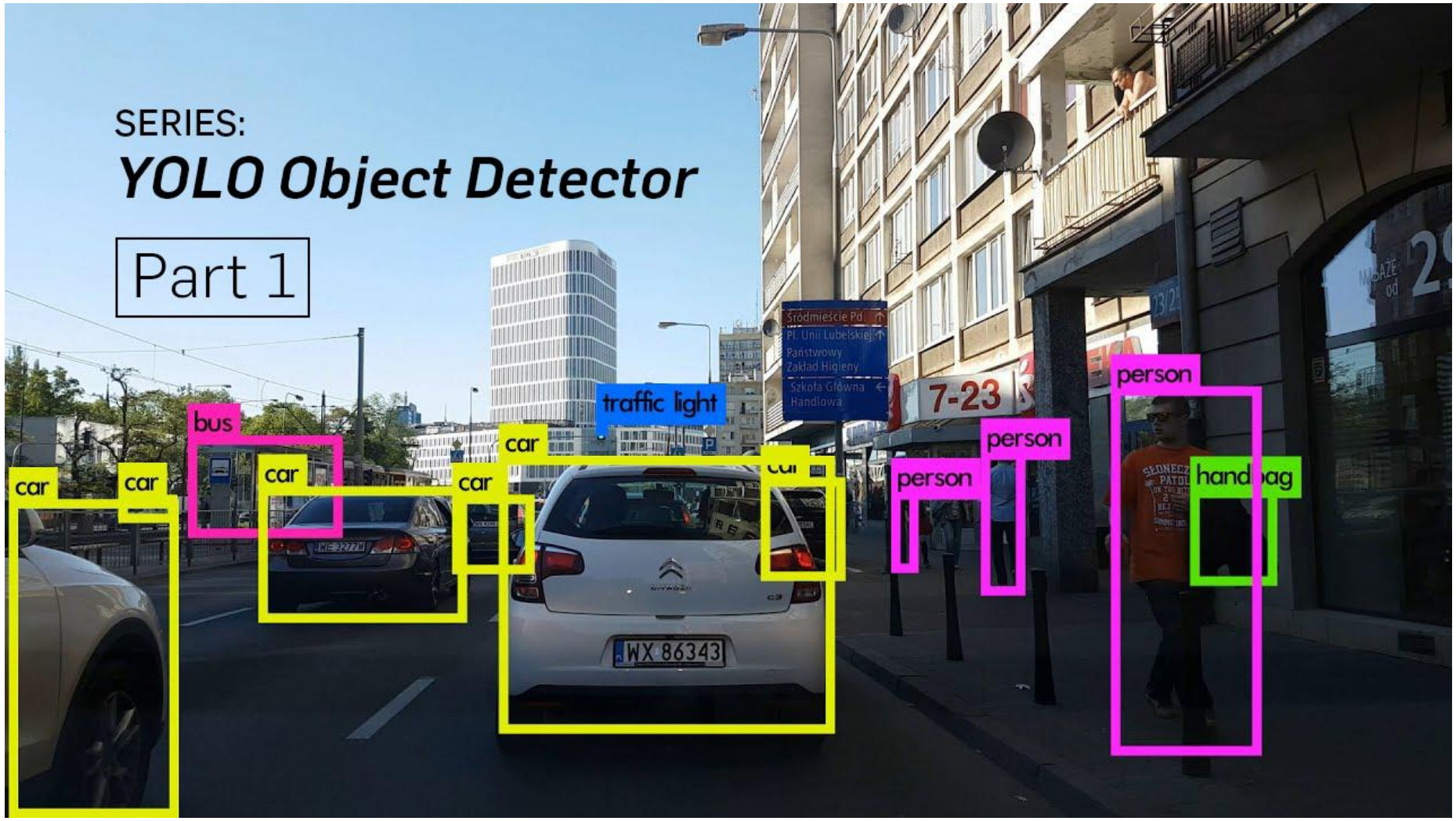
YOLOv1

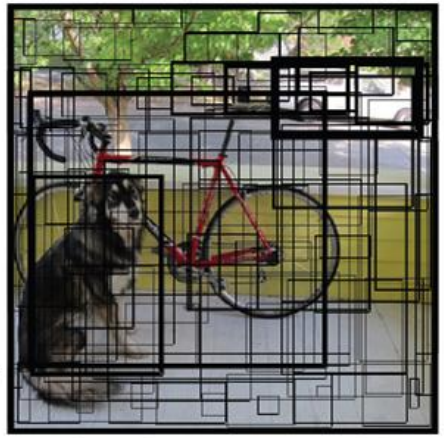
從原理開始理解
大家在講的yolo是啥

這堂課第一個介紹會給你框的神經網路

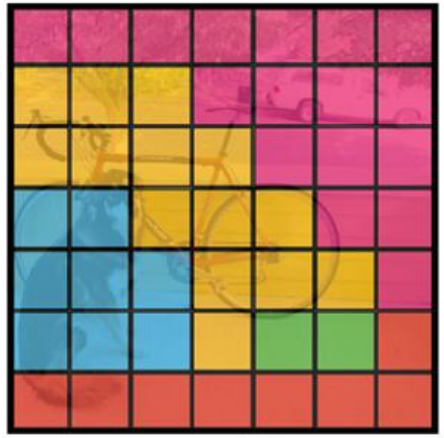


大致原理

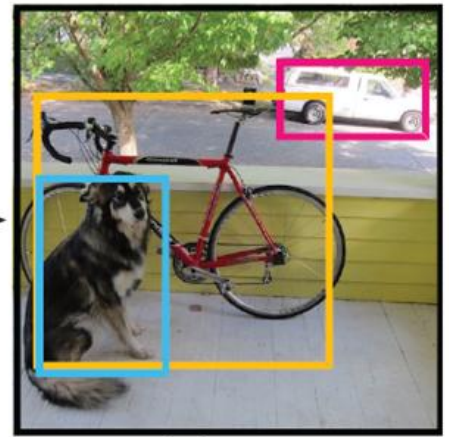
切成幾成幾的格子



輸出一堆的框框



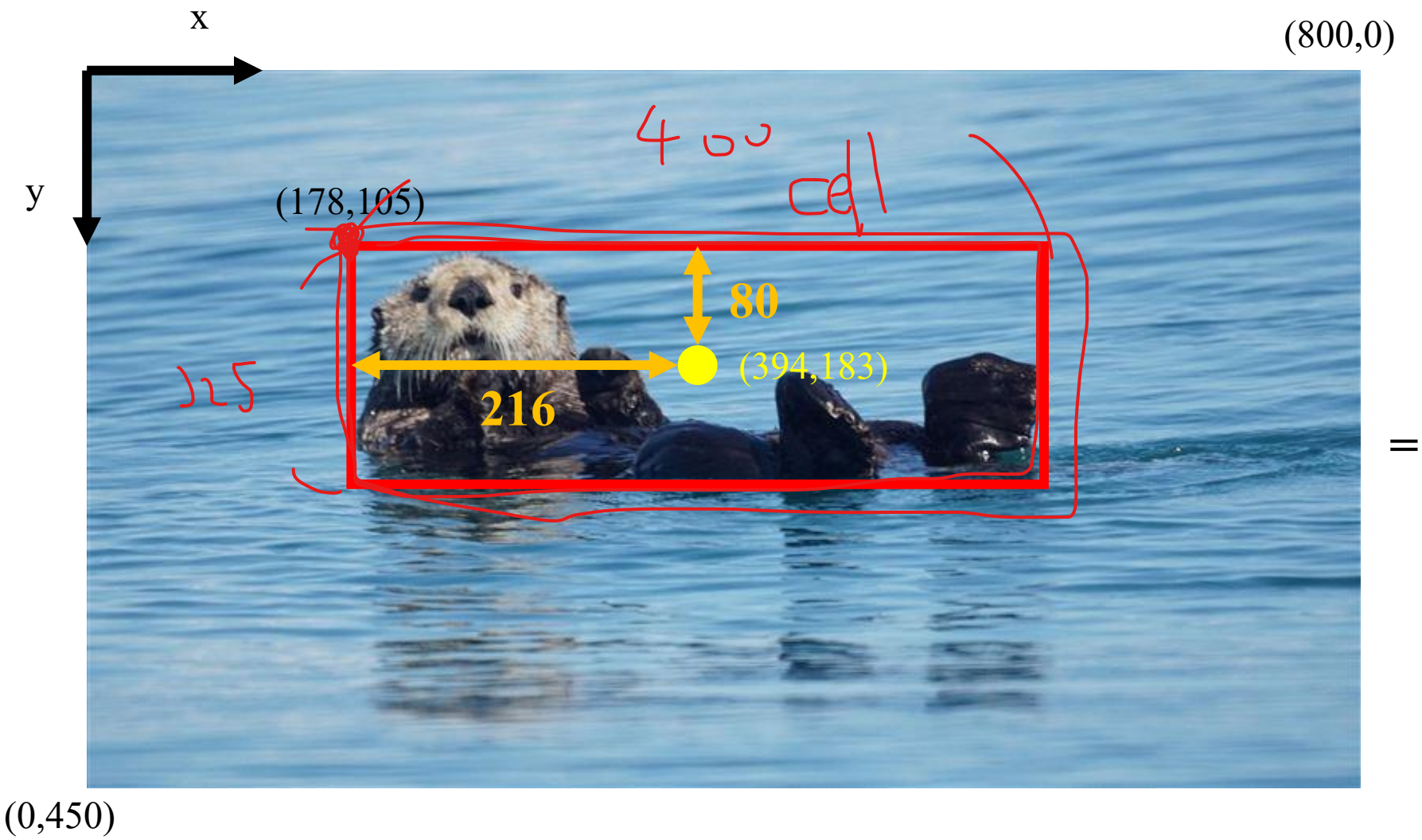
每格只會有一個信心最高的類別



過濾分數低的框，
並且看框的中心落在哪個類別

原理細節

輸出長怎樣?



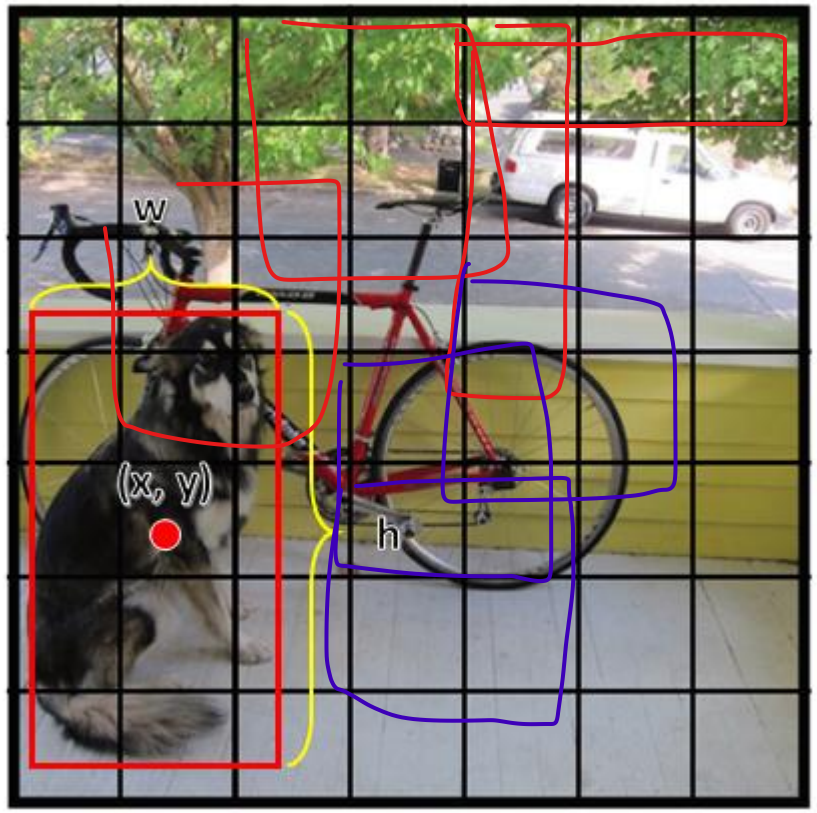
$$
\begin{bmatrix} P_c \\ B_x \\ B_y \\ B_w \\ B_h \\ C_1 \\ C_2 \end{bmatrix}
=
\begin{bmatrix} 1 \\ \cancel{394} \\ \cancel{183} \\ \cancel{216} \\ \cancel{80} \\ 1 \\ 0 \end{bmatrix}
\begin{matrix} \\ 216 \\ 80 \\ 0.5 \\ 0.5 \\ \\ \\ \end{matrix}
$$

C1=海獺
C2=狗

原論文輸出是7*7*30

7*7是因為圖片分成7*7的區域

30是因為5+5+20



一個格子可以輸出兩種東西

(p,x,y,w,h) (p,x,y,w,h) 資料集有20個類別

水 土

(x, y, w, h)	C	(x, y, w, h)	C	$(c_1, c_2, \ldots, c_{20})$

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

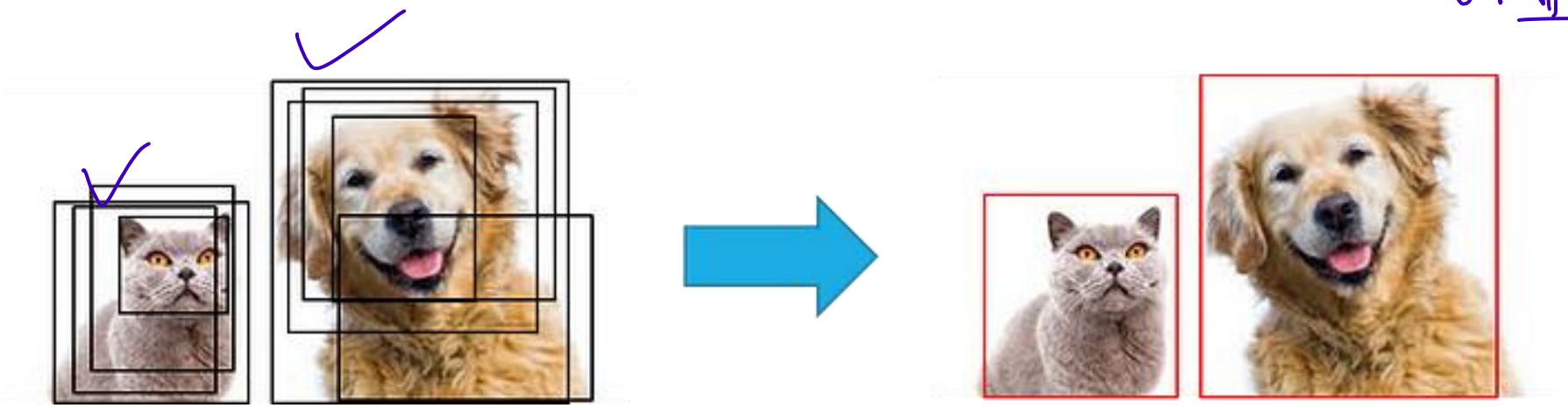
$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

框 *2

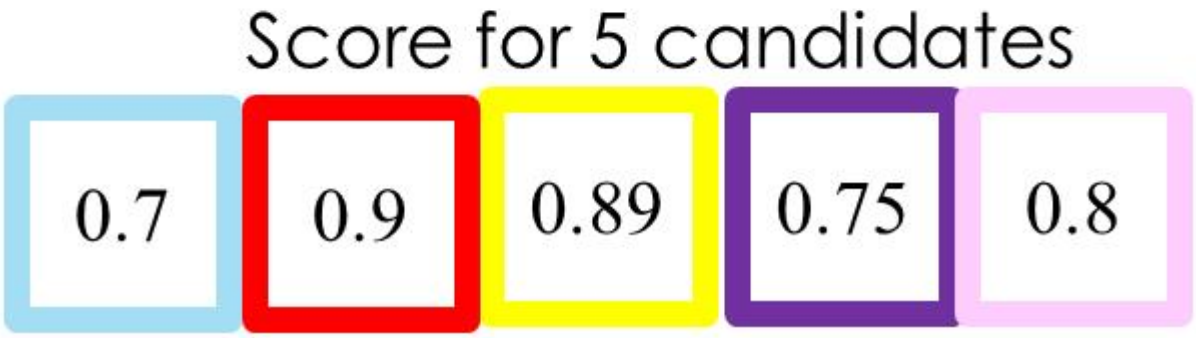
信心度

Non-Maximum Suppression(NMS) 非極大值抑制

threshold → 閾值



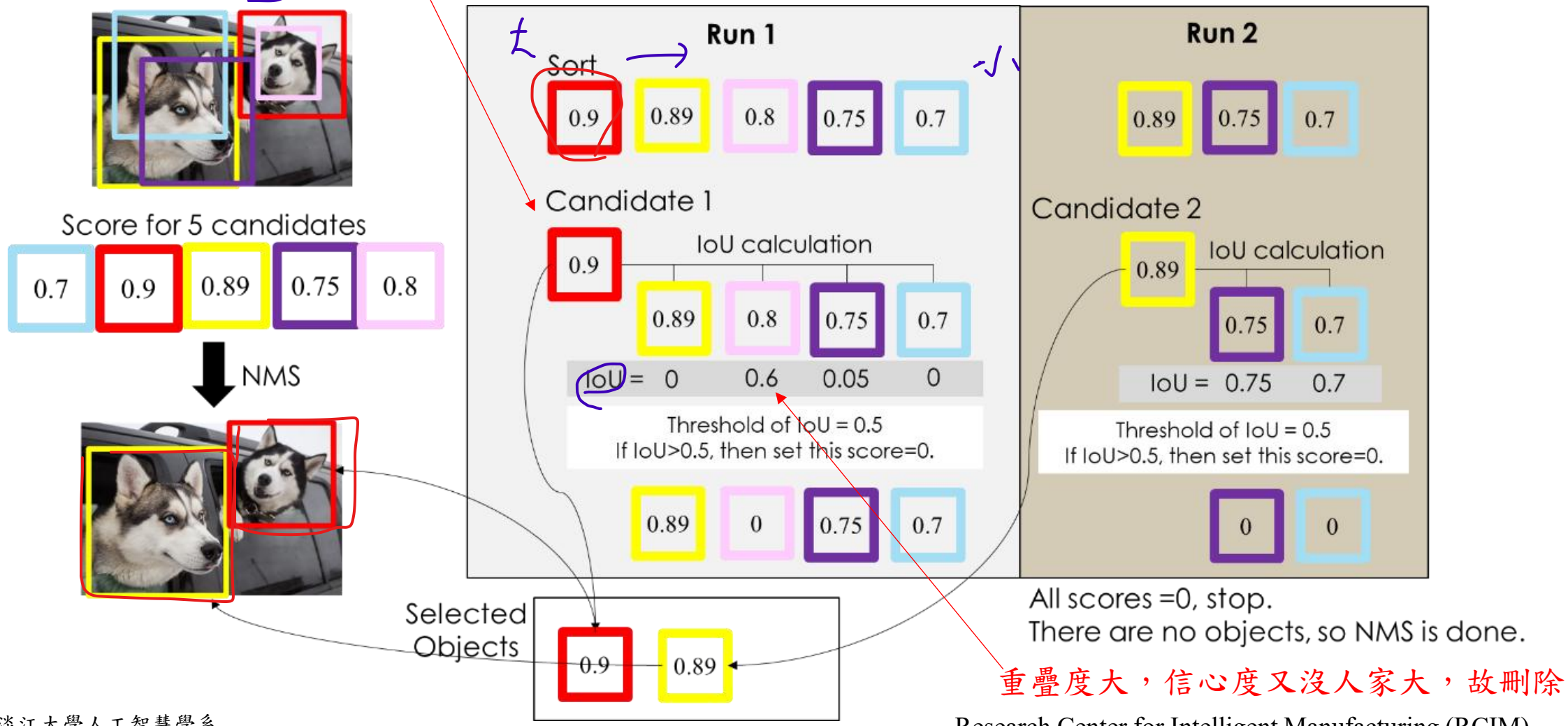
Non-Maximum Suppression(NMS) 非極大值抑制



我們假設這張圖我們捉到5個BBox，每個BBox帶有一個是不是物件的信心。

NMS處理一種類別時的步驟

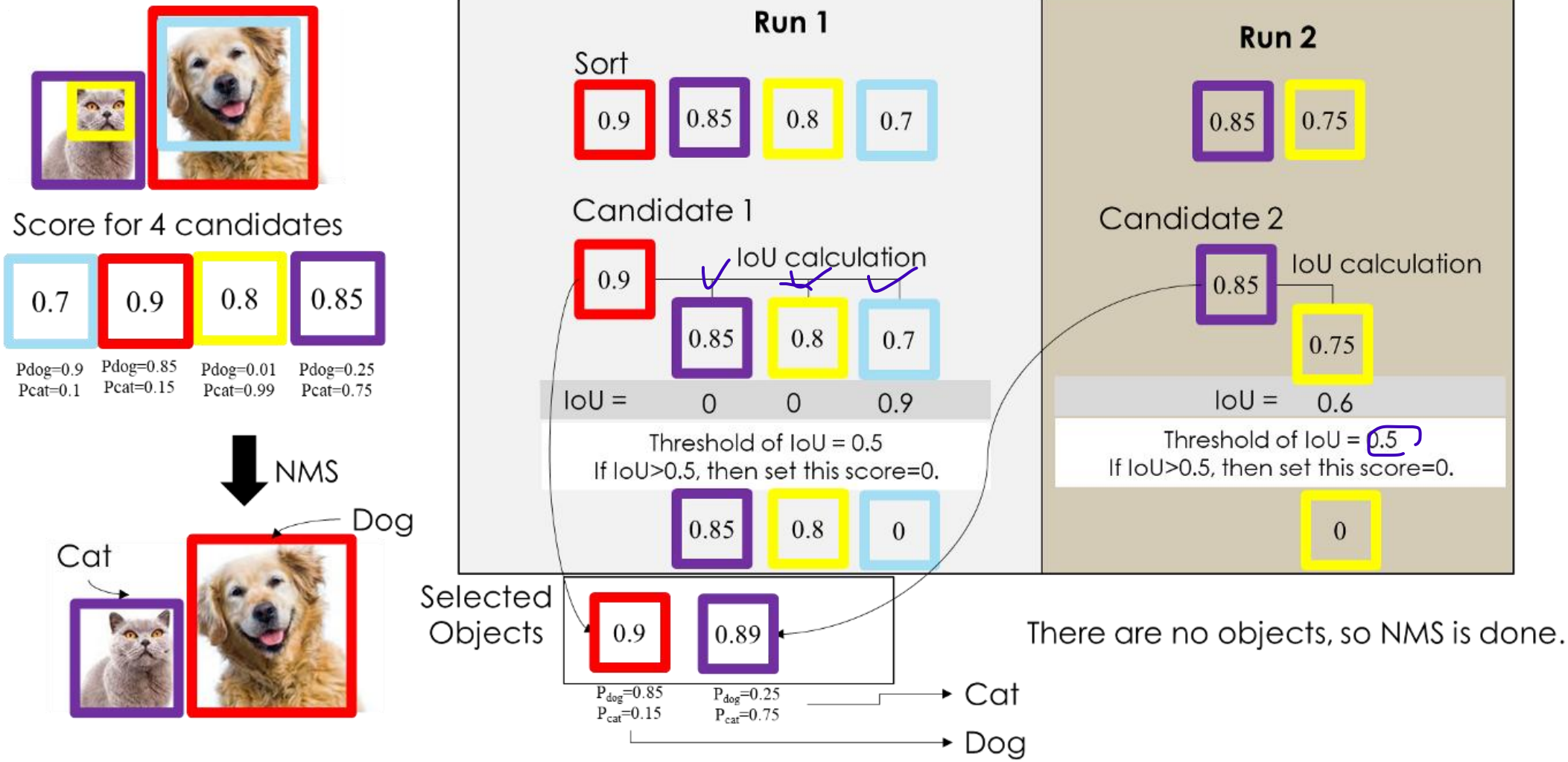
挑選信心度最大的開始比對



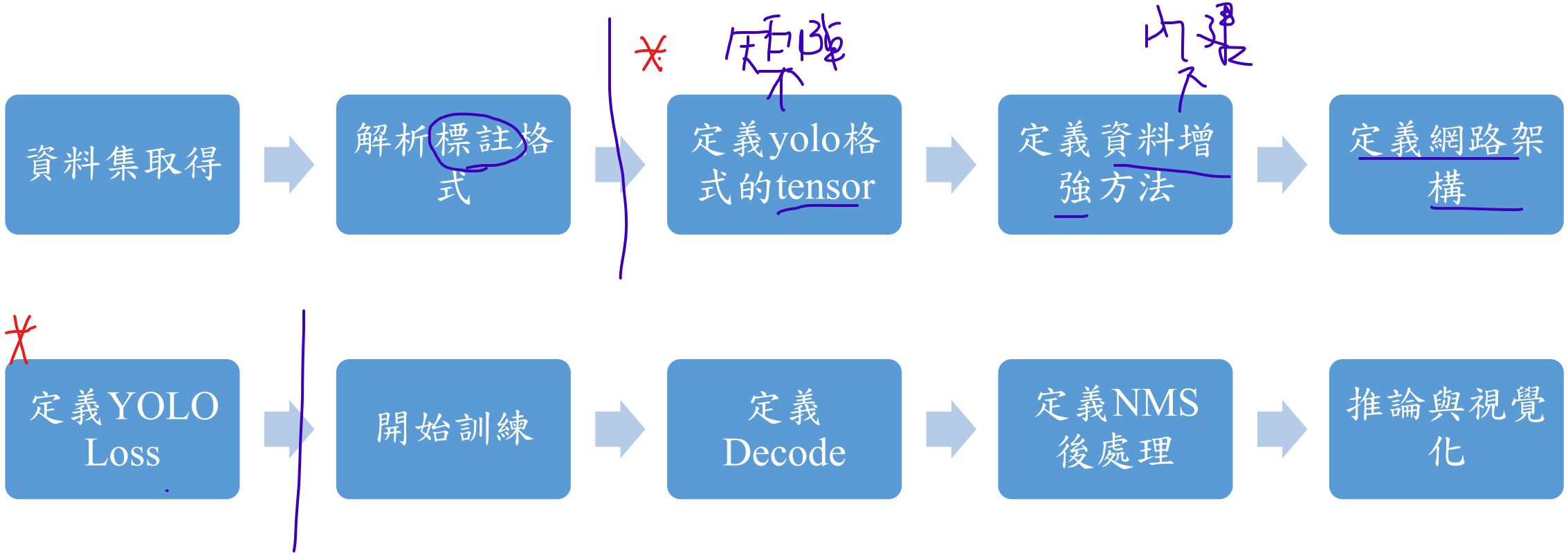
重疊度大，信心度又沒人家大，故刪除

NMS處理多種類別時的步驟

先將「確定是物件集合」選出來，再搭配一下分類的機率



YOLO整體步驟


資料集取得
解析標註格式
定義yolo格式的tensor
定義資料增強方法
定義網路架構
定義YOLO Loss
開始訓練
定義Decode
定義NMS後處理
推論與視覺化

程式

這次的程式炸多

他們都很重要。如果沒有手打，很大機率會不懂YOLO

資料集取得

資料集載入

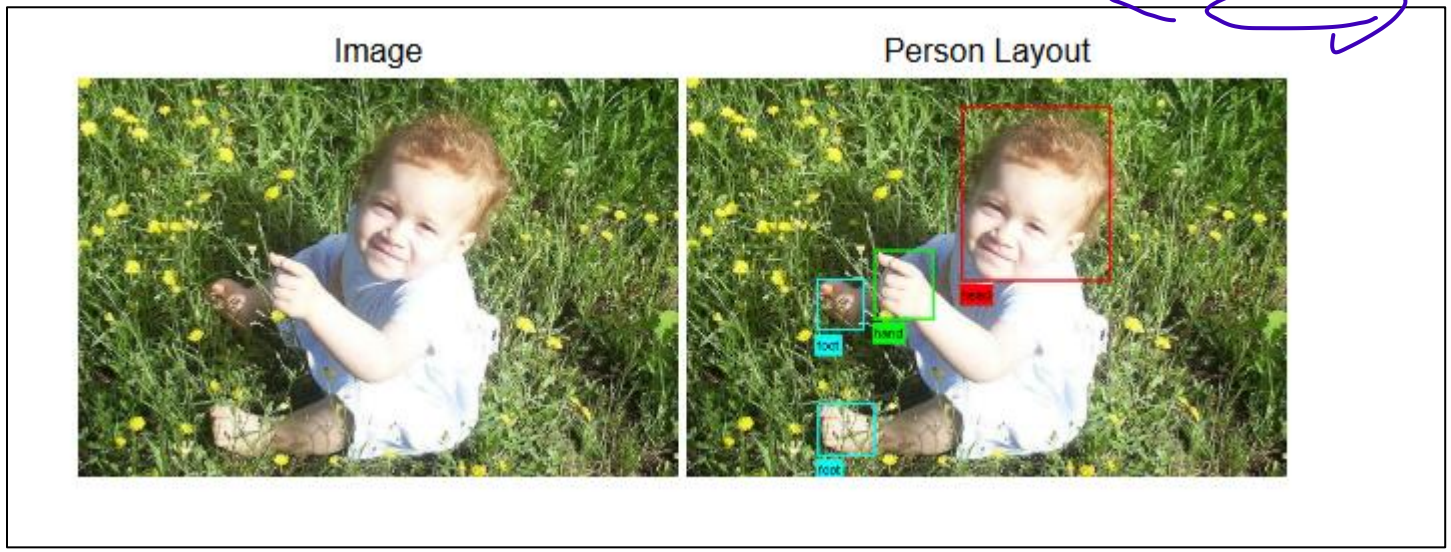
- 使用VOC 2007（共20類）

train

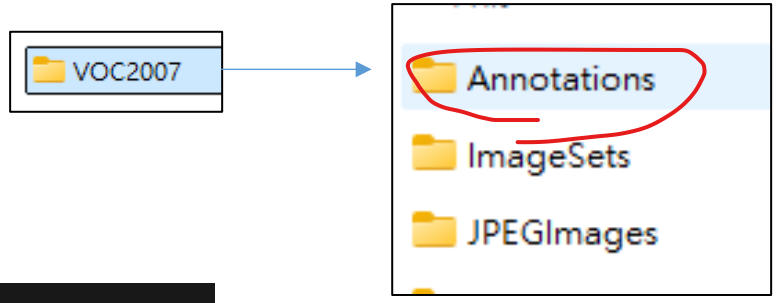
```
VOCdevkit/VOC2007/
|—Annotations/ <-XML標註
|—ImageSets/Main/ <-train.txt / val.txt / test.txt
|_JEPGImages/ <-原始圖片
```

圖片本人

存放位置



資料集載入



- 使用VOC 2007（共20類）

run(list)

```
import subprocess #用來處理所有os指令的模組
import tarfile

DATA_ROOT = Path('./data')
DATA_ROOT.mkdir(exist_ok=True)
tar_file = ['VOCtrainval_06-Nov-2007.tar', 'VOCtest_06-Nov-2007.tar']

VOC_DIR = DATA_ROOT / 'VOCdevkit' / 'VOC2007'

if not VOC_DIR.exists():

    for tar_name in tar_file:
        print(f"下載檔案 -> {tar_name}")
        subprocess.run([
        'curl', '-L',
        f'http://host.robots.ox.ac.uk/pascal/VOC/voc2007/{tar_name}',
        '--output',str(DATA_ROOT / tar_name)
        ], check=True)

        print("--- 開始解壓縮... ---")
        tar_path = DATA_ROOT / tar_name
        with tarfile.open(tar_path) as F:
            F.extractall(path=DATA_ROOT)
        print(f"->{tar_name}解壓縮完成，目的地{DATA_ROOT}")
else:
    print("資料集存在，略過下載步驟!")
```

下載Dic

資料集載入

- 引用subprocess模組 + tarfile模組

原指令：curl –L http://host.robots.ox.ac.uk/pascal/VOC/voc2007/VOCtrainval_06-Nov-2007.tar --output ./data/VOCtrainval_06-Nov-2007.tar

```
subprocess.run([
'curl', '-L',
f'http://host.robots.ox.ac.uk/pascal/VOC/voc2007/{tar_name}',
'--output',str(DATA_ROOT / tar_name)
], check=True)
```

解壓縮tar檔

```
with tarfile.open(tar_path) as F:
    F.extractall(path=DATA_ROOT)
print(f"->{tar_name}解壓縮完成，目的地{DATA_ROOT}")
```

解析標註格式

資料集標記

- Annotation 為 XML 的格式

```
<annotation>
    <folder>VOC2007</folder>
    <filename>000001.jpg</filename>
    <source>
        <database>The VOC2007 Database</database>
        <annotation>PASCAL VOC2007</annotation>
        <image>flickr</image>
        <flickrid>341012865</flickrid>
    </source>
    <owner>
        <flickrid>Fried Camels</flickrid>
        <name>Jinky the Fruit Bat</name>
    </owner>
    <size>
        <width>353</width>
        <height>500</height>
        <depth>3</depth>
    </size>
```

Class idx

img_w,
img_h

```
    </size>
    <segmented>0</segmented>
    <object>
        <name>dog</name>
        <pose>Left</pose>
        <truncated>1</truncated>
        <difficult>0</difficult>
        <bndbox>
            <xmin>48</xmin>
            <ymin>240</ymin>
            <xmax>195</xmax>
            <ymax>371</ymax>
        </bndbox>
    </object>
    <object>
        <name>person</name>
        <pose>Left</pose>
        <truncated>1</truncated>
        <difficult>0</difficult>
        <bndbox>
            <xmin>8</xmin>
            <ymin>12</ymin>
            <xmax>352</xmax>
            <ymax>498</ymax>
        </bndbox>
    </object>
</annotation>
```

Xmin,ymin,
Xmax,ymax

資料集標記

- Annotation 為 XML 的格式

Step1. 定義20個類別

```python
VOC_CLASSES = [
    'aeroplane', 'bicycle', 'bird', 'boat', 'bottle',
    'bus', 'car', 'cat', 'chair', 'cow',
    'diningtable', 'dog', 'horse', 'motorbike', 'person',
    'pottedplant', 'sheep', 'sofa', 'train', 'tvmonitor'
]
NUM_CLASSES = len(VOC_CLASSES)  # 20
CLASS2IDX = {cls: i for i, cls in enumerate(VOC_CLASSES)}

print(f'類別數量: {NUM_CLASSES}')
print('類別列表:', CLASS2IDX)
```

```
類別數量: 20
類別列表: {'aeroplane': 0, 'bicycle': 1, 'bird': 2, 'boat': 3, 'bottle': 4, 'bus': 5, 'car': 6, 'cat
```

資料集標記

- Annotation 為 XML 的格式

Step2. 從標記拿取 (class_idx, x1, y1, x2, y2, W, H)

```python
import xml.etree.ElementTree as ET
def parse_voc_xml(xml_path):
    """
    解析 VOC XML 標注檔
    回傳: [(class_idx, x_min, y_min, x_max, y_max), ...]
    座標為像素絕對值
    """
    tree = ET.parse(xml_path)
    root = tree.getroot()

    # 圖片尺寸
    size = root.find('size')
    img_w = int(size.find('width').text)
    img_h = int(size.find('height').text)

```

str

```python
    boxes = []
    for obj in root.findall('object'):
        cls_name = obj.find('name').text.strip()
        if cls_name not in CLASS2IDX:
            continue
        difficult = int(obj.find('difficult').text)
        if difficult:
            continue  # 略過困難樣本

        bbox = obj.find('bndbox')
        x1 = float(bbox.find('xmin').text)
        y1 = float(bbox.find('ymin').text)
        x2 = float(bbox.find('xmax').text)
        y2 = float(bbox.find('ymax').text)

        boxes.append((CLASS2IDX[cls_name], x1, y1, x2, y2, img_w, img_h))

    return boxes

# 測試解析八個 XML
sample_xml = list((VOC_DIR / 'Annotations').glob('*.xml'))[8]
sample_boxes = parse_voc_xml(sample_xml)
print(f'範例標注檔: {sample_xml.name}')
print(f'標注數量: {len(sample_boxes)}')
print("格式 (class_idx, x1, y1, x2, y2, W, H)")
for i in sample_boxes:
    print(f'- {i}')
```

(x_1, y_1) (x_2, y_2)

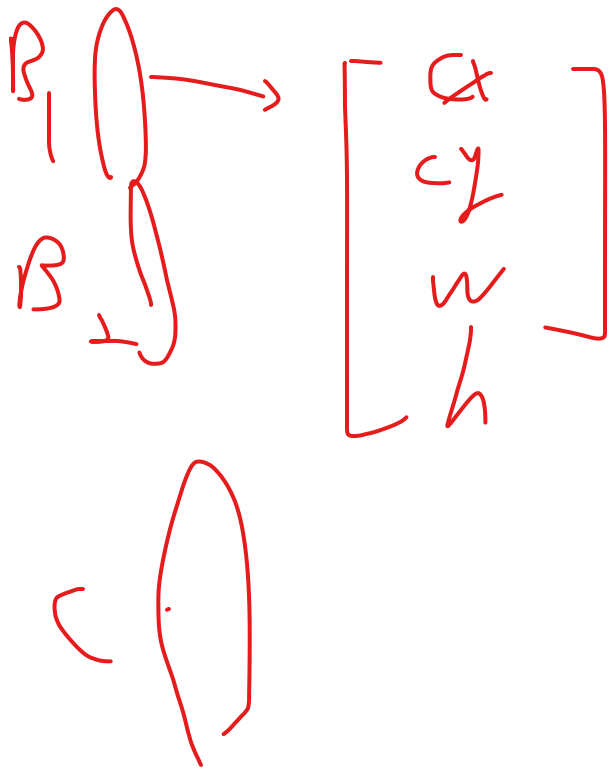
tensor

```
範例標注檔: 000009.xml
標注數量: 4
格式 (class_idx, x1, y1, x2, y2, W, H)
- (12, 69.0, 172.0, 270.0, 330.0, 500, 375)
- (14, 150.0, 141.0, 229.0, 284.0, 500, 375)
- (14, 285.0, 201.0, 327.0, 331.0, 500, 375)
- (14, 258.0, 198.0, 297.0, 329.0, 500, 375)
```

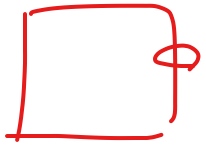
定義Yolo格式tensor

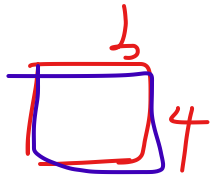
Yolo格式

- 圖片會分割成 SxS = 7x7的格子，每個格子負責預測:
- B=2個Bounding Box: (x,y,w,h,confidence)
- c=20個類別機率




```python
S = 7 #Grid數量 (7x7)
B = 2 #每個cell預測的bounding box數量
C = 20 #class數 (pascal voc 有 20 類)

def encode_yolo_label(boxes, img_size=448):
    """
    將VOC格式轉為Yolov1 tensor
    輸入 : boxes[(class_idx, x1, y1, x2, y2,w, h),....]
    輸出 : target: [7, 7, 30], target[i, j] = [x, y, w, h, conf, x, y, w, h, conf, cls0, cls1,..., cls19]
    """

    target = torch.zeros(S, S, B*5+C)

    for cls_idx, x1, y1, x2, y2, orig_W, orig_H in boxes:

        # 步驟1. 將座標縮放至 448x448
        x1 = x1 / orig_W * img_size
        y1 = y1 / orig_H * img_size
        x2 = x2 / orig_W * img_size
        y2 = y2 / orig_H * img_size
```





Yolo格式

S = 7 x 7

```python
# 步驟2. 計算物體中心位置
        cx = (x1 + x2) / 2
        cy = (y1 + y2) / 2
        w = (x2 - x1)
        h = (y2 - y1)

        # 步驟3. 找到物體中心落在哪個cell中
        cell_size = img_size / S
        col = int(cx / cell_size)
        row = int(cy / cell_size)
        col = min(col, S - 1) #處理邊界溢出 cx=448 / 64 = 7 會溢出
        row = min(row, S - 1)

        # 步驟4. 編碼座標
        # x,y相對於所在 cell 左上角的偏移，歸一化到 [0,1]
        # cx=300, cx/cell_size=300/64 = 4.6875
        # 4.6875 - 4 = 0.6875 (所以會落在第四格偏右68%)
        x_cell = (cx / cell_size) - col
        y_cell = (cy / cell_size) - row
        # w, h：相對於整張圖的比例
        w_norm = w / img_size
        h_norm = h / img_size
```

448/7 = 64

(7+是1)

限一個cell → 1 class

```python
        # 步驟5. 如果這個cell沒有物體，填入標註
        if target[row, col, 4] == 0:  # conf == 0 代表還沒有物體
            # bbox 1 (indices 0-4)
            target[row, col, 0] = x_cell
            target[row, col, 1] = y_cell
            target[row, col, 2] = w_norm
            target[row, col, 3] = h_norm
            target[row, col, 4] = 1.0  # confidence = 1

            # bbox 2 (indices 5-9) — 初始與 bbox1 相同
            target[row, col, 5] = x_cell
            target[row, col, 6] = y_cell
            target[row, col, 7] = w_norm
            target[row, col, 8] = h_norm
            target[row, col, 9] = 1.0

            # 類別 one-hot (indices 10-29)
            target[row, col, B * 5 + cls_idx] = 1.0

    return target
print(f'輸出 Target 形狀: [{S}, {S}, {B*5+C}]')
```

B_1

B_2

```
輸出 Target 形狀: [7, 7, 30]
```

定義資料增強方法

資料增強(論文)

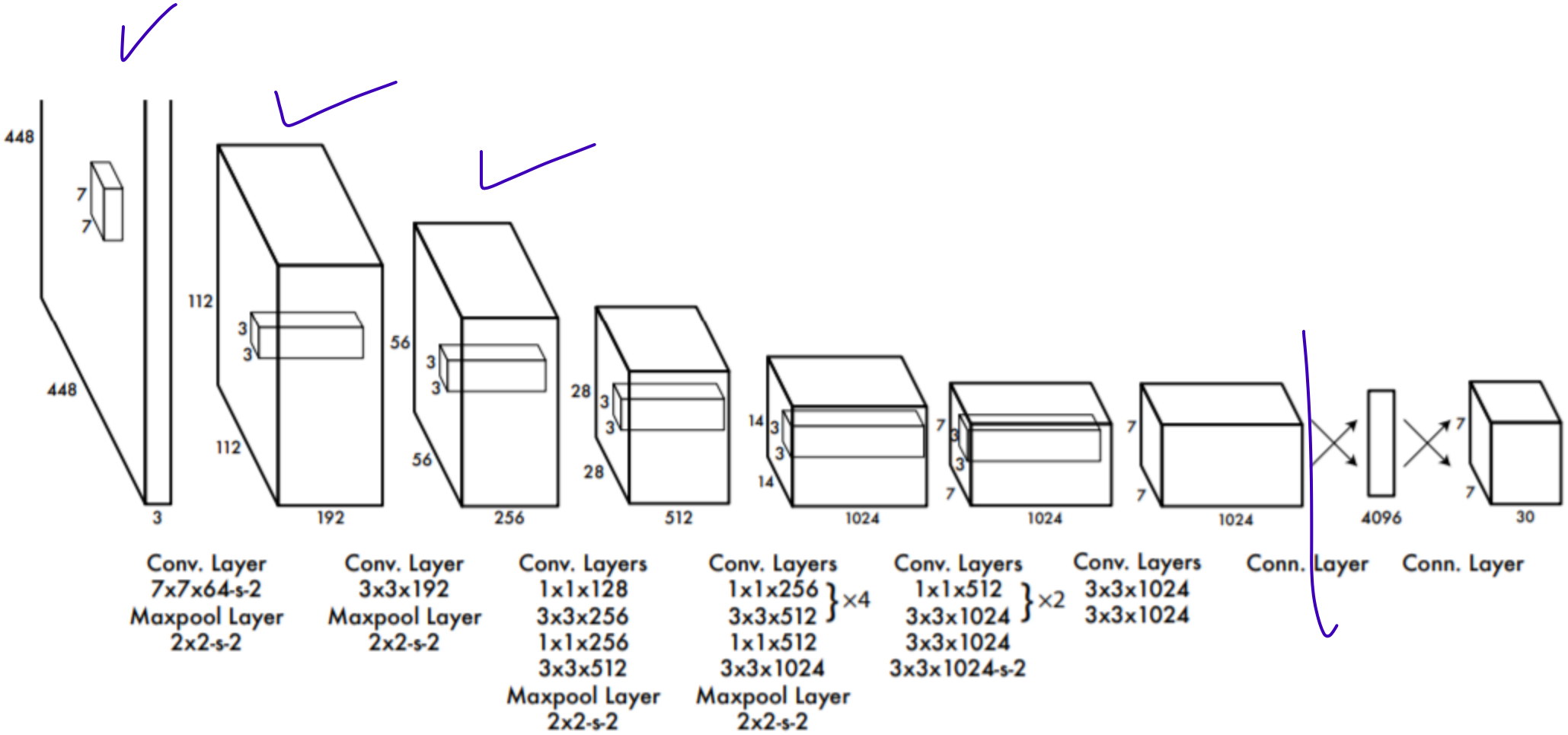
- 隨機縮放 ✓
- 隨機水平翻轉 → 排人
- HSV色彩空間調整
- 規一化 [0,1]

```python
if self.augment and torch.rand(1).item() > 0.5:
    image = T.functional.hflip(image)
    orig_W = image.width
    flipped = []
    for cls_idx, x1, y1, x2, y2, W, H in boxes:
        new_x1 = W-x2
        new_x2 = W-x1
        flipped.append((cls_idx, new_x1, y1, new_x2, y2, W, H))
    boxes = flipped
```

```python
self.base_transform = T.Compose([
    T.Resize((img_size, img_size)),
    T.ToTensor(),
    T.Normalize(
        mean=[0.485, 0.456, 0.406],
        std =[0.229, 0.224, 0.225]
    )
])
```

定義網路架構

原論文架構


448
7
7
448
3
112
3
3
112
192
56
3
3
56
256
28
3
3
28
512
14
3
3
14
1024
7
3
3
7
1024
7
7
1024
4096
7
7
30
Conv. Layer
7x7x64-s-2
Maxpool Layer
2x2-s-2
Conv. Layer
3x3x192
Maxpool Layer
2x2-s-2
Conv. Layers
1x1x128
3x3x256
1x1x256
3x3x512
Maxpool Layer
2x2-s-2
Conv. Layers
1x1x256
3x3x512
×4
1x1x512
3x3x1024
Maxpool Layer
2x2-s-2
Conv. Layers
1x1x512
3x3x1024
×2
3x3x1024
3x3x1024-s-2
Conv. Layers
3x3x1024
3x3x1024
Conn. Layer
Conn. Layer

定義Yolo網路

```python
class ConvBnLReLU(nn.Module):
    def __init__(self, in_ch, out_ch, kernel_size, stride=1, padding=0):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size, stride, padding, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.LeakyReLU(0.1, inplace=True)
        )
    def forward(self, x):
        return self.block(x)
```

定義Yolo網路(2)

```python
class YOLOv1(nn.Module):
    def __init__(self, S=7, B=2, C=20):
        super().__init__()
        self.S = S
        self.B = B
        self.C = C

        self.features = nn.Sequential(
            # Block 1: 448→224
            ConvBnLReLU(3,   64,  7, stride=2, padding=3),   # Conv1
            nn.MaxPool2d(2, 2),

            # Block 2: 224→112
            ConvBnLReLU(64,  192, 3, padding=1),             # Conv2
            nn.MaxPool2d(2, 2),

            # Block 3: 112→56 (Inception-like, 4 layers)
            ConvBnLReLU(192, 128, 1),                        # Conv3
            ConvBnLReLU(128, 256, 3, padding=1),             # Conv4
            ConvBnLReLU(256, 256, 1),                        # Conv5
            ConvBnLReLU(256, 512, 3, padding=1),             # Conv6
            nn.MaxPool2d(2, 2),

```

```python
            # Block 4: 56→28 (4×{1×1, 3×3} + 2 extra)
            ConvBnLReLU(512, 256, 1),
            ConvBnLReLU(256, 512, 3, padding=1),
            ConvBnLReLU(512, 256, 1),
            ConvBnLReLU(256, 512, 3, padding=1),
            ConvBnLReLU(512, 256, 1),
            ConvBnLReLU(256, 512, 3, padding=1),
            ConvBnLReLU(512, 256, 1),
            ConvBnLReLU(256, 512, 3, padding=1),
            ConvBnLReLU(512, 512, 1),
            ConvBnLReLU(512, 1024, 3, padding=1),
            nn.MaxPool2d(2, 2),

            # Block 5: 28→14 (2×{1×1, 3×3} + 2 extra)
            ConvBnLReLU(1024, 512,  1),
            ConvBnLReLU(512,  1024, 3, padding=1),
            ConvBnLReLU(1024, 512,  1),
            ConvBnLReLU(512,  1024, 3, padding=1),
            ConvBnLReLU(1024, 1024, 3, padding=1),
            ConvBnLReLU(1024, 1024, 3, stride=2, padding=1),  # → 7×7

            # Block 6: 7×7
            ConvBnLReLU(1024, 1024, 3, padding=1),
            ConvBnLReLU(1024, 1024, 3, padding=1),
        )
```

定義Yolo網路(3)

```python
self._init_weights()

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, a=0.1)
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.ones_(m.weight)
                nn.init.zeros_(m.bias)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                nn.init.zeros_(m.bias)
    def forward(self, x):
        x = self.features(x)
        x = self.head(x)
        x = x.view(-1, self.S, self.S, self.B * 5 + self.C)
        return x
```

定義YOLO Loss

Intersection over Union(IOU)

重疊性

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

→ 共有

→ 聯合

定義IOU

```python
def compute_iou(box1, box2):
    # 轉換為左上/右下角格式
    b1_x1 = box1[..., 0] - box1[..., 2] / 2
    b1_y1 = box1[..., 1] - box1[..., 3] / 2
    b1_x2 = box1[..., 0] + box1[..., 2] / 2
    b1_y2 = box1[..., 1] + box1[..., 3] / 2

    b2_x1 = box2[..., 0] - box2[..., 2] / 2
    b2_y1 = box2[..., 1] - box2[..., 3] / 2
    b2_x2 = box2[..., 0] + box2[..., 2] / 2
    b2_y2 = box2[..., 1] + box2[..., 3] / 2

    # 交集
    inter_x1 = torch.max(b1_x1, b2_x1)
    inter_y1 = torch.max(b1_y1, b2_y1)
    inter_x2 = torch.min(b1_x2, b2_x2)
    inter_y2 = torch.min(b1_y2, b2_y2)

    inter_area = (inter_x2 - inter_x1).clamp(0) * (inter_y2 - inter_y1).clamp(0)

    # 聯集
    area1 = (b1_x2 - b1_x1) * (b1_y2 - b1_y1)
    area2 = (b2_x2 - b2_x1) * (b2_y2 - b2_y1)
    union_area = area1 + area2 - inter_area + 1e-6

    return inter_area / union_area
```

B_1

B_2

防止出現

原論文Loss定義

λ_coord = 5

λ_noobj = 0.5

中心座標Loss

位置

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

寬高 Loss

$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

有物體的置信度 Loss

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2$$

無物體的置信度 Loss

$$+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2$$

0 1

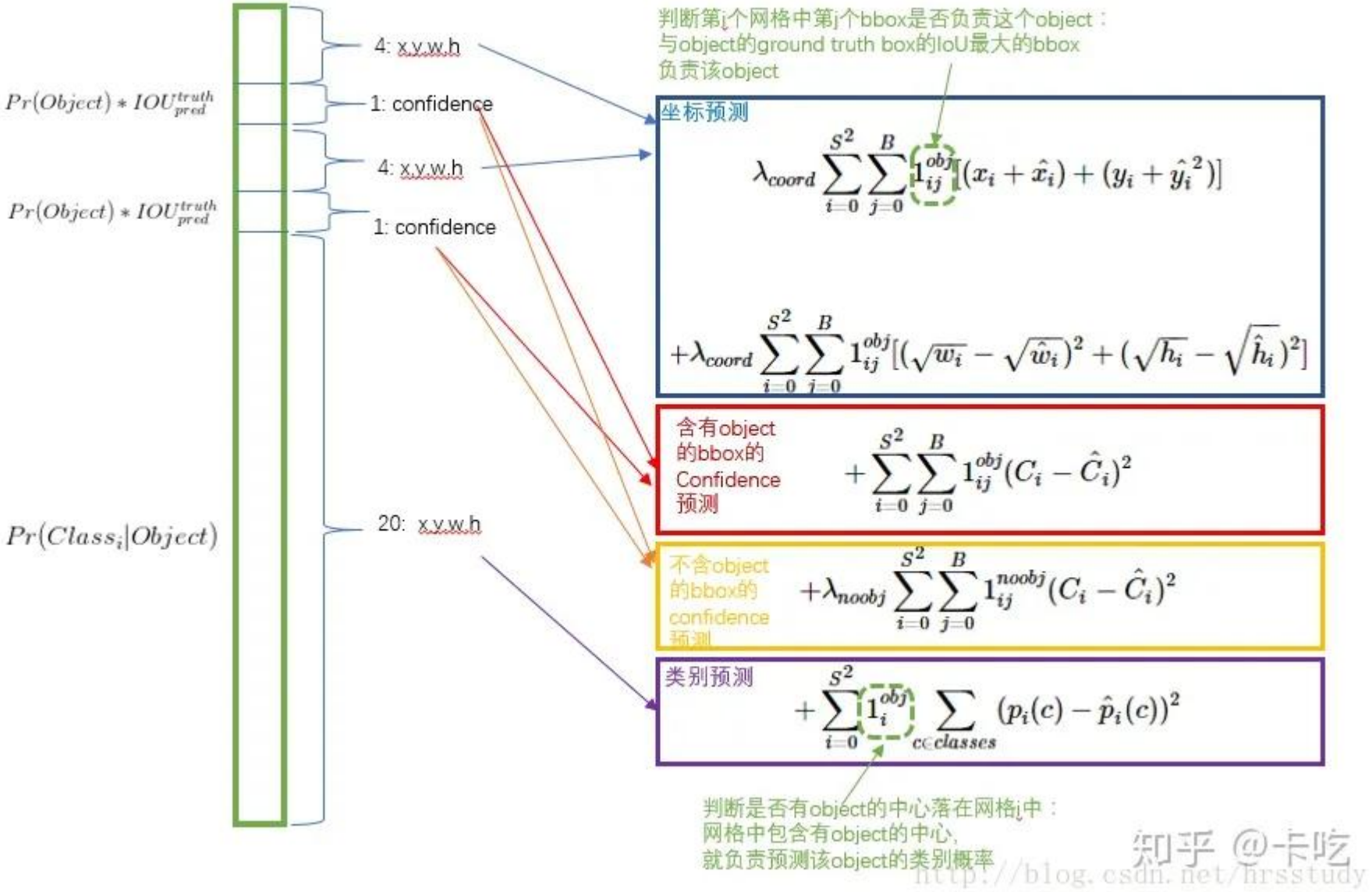
48 X

1 V

$$+ \sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} \left(p_i(c) - \hat{p}_i(c) \right)^2 \quad (3)$$

類別 Loss

全是MSE


判断第i个网格中第j个bbox是否负责这个object：
与object的ground truth box的IoU最大的bbox
负责该object
4: x,y,w,h
1: confidence
4: x,y,w,h
1: confidence
20: x,y,w,h
$Pr(Object) * IOU_{pred}^{truth}$
$Pr(Object) * IOU_{pred}^{truth}$
$Pr(Class_i|Object)$
坐标预测
$\lambda_{coord}\sum_{i=0}^{S^2}\sum_{j=0}^{B}1_{ij}^{obj}[(x_i+\hat{x}_i)+(y_i+\hat{y}_i{}^2)]$
$+\lambda_{coord}\sum_{i=0}^{S^2}\sum_{j=0}^{B}1_{ij}^{obj}[(\sqrt{w_i}-\sqrt{\hat{w}_i})^2+(\sqrt{h_i}-\sqrt{\hat{h}_i})^2]$
含有object的bbox的Confidence预测
$+\sum_{i=0}^{S^2}\sum_{j=0}^{B}1_{ij}^{obj}(C_i-\hat{C}_i)^2$
不含object的bbox的confidence预测
$+\lambda_{noobj}\sum_{i=0}^{S^2}\sum_{j=0}^{B}1_{ij}^{noobj}(C_i-\hat{C}_i)^2$
类别预测
$+\sum_{i=0}^{S^2}1_i^{obj}\sum_{c\in classes}(p_i(c)-\hat{p}_i(c))^2$
判断是否有object的中心落在网格i中：
网格中包含有object的中心，
就负责预测该object的类别概率
知乎 @卡吃
http://blog.csdn.net/hrsstudy

Loss(1)

```python
class YOLOv1Loss(nn.Module):
    def __init__(self, S=7, B=2, C=20, lambda_choord=5.0, lambda_noobj=0.5):
        super().__init__()
        self.S = S
        self.B = B
        self.C = C
        self.lambda_coord = lambda_choord #座標loss權重
        self.lambda_noobj = lambda_noobj #無物體 loss權重
        self.mse = nn.MSELoss(reduction='sum')

    def forward(self, predictions, targets):
        batch_size = predictions.shape[0]

        # 拆解預測 & 目標
        pred_b1   = predictions[..., 0:5]
        pred_b2   = predictions[..., 5:10]
        pred_cls  = predictions[..., 10:]

        tgt_b1    = targets[..., 0:5]
        tgt_b2    = targets[..., 5:10] #跟tgt_b1是一樣的值
        tgt_cls   = targets[..., 10:]

        # obj mask:有物體的cell位置 , noobj:沒有物體
        obj_mask = targets[...,4]
        noobj_mask = 1 - obj_mask

        # 找 Responsible box (IoU最高的 box) -> 話語權
        iou_b1 = compute_iou(pred_b1[..., :4], tgt_b1[..., :4])
        iou_b2 = compute_iou(pred_b2[..., :4], tgt_b1[..., :4])

        # responsible_b1[i,j]: True 代表 box1 的 IoU 較高
        responsible_b1 = (iou_b1 >= iou_b2).float()  # [B, S, S]
        responsible_b2 = 1 - responsible_b1
```

預

真

比出一個?

比IoU

Loss(2)

```python
# ── ① + ② 座標 Loss（只計算 responsible & 有物體的格子）─
obj_b1 = obj_mask * responsible_b1  # [B, S, S]
obj_b2 = obj_mask * responsible_b2

# 中心座標 loss
xy_loss = (
    obj_b1.unsqueeze(-1) * (pred_b1[..., :2] - tgt_b1[..., :2]).pow(2) +
    obj_b2.unsqueeze(-1) * (pred_b2[..., :2] - tgt_b1[..., :2]).pow(2)
).sum()

# 寬高 loss（開根號！小物體更重要）
# clamp 避免負數開根號 、 設定最小值1e-6避免梯度爆炸
wh_loss = (
    obj_b1.unsqueeze(-1) * (
        pred_b1[..., 2:4].clamp(min=1e-6).sqrt() -
        tgt_b1[..., 2:4].clamp(min=1e-6).sqrt()
    ).pow(2) +
    obj_b2.unsqueeze(-1) * (
        pred_b2[..., 2:4].clamp(min=1e-6).sqrt() -
        tgt_b1[..., 2:4].clamp(min=1e-6).sqrt()
    ).pow(2)
).sum()

coord_loss = self.lambda_coord * (xy_loss + wh_loss)

# ── ③ 有物體的置信度 Loss ──────────────────────
conf_obj_loss = (
    obj_b1 * (pred_b1[..., 4] - iou_b1.detach()).pow(2) +
    obj_b2 * (pred_b2[..., 4] - iou_b2.detach()).pow(2)
).sum()
```

Loss(3)

```python
# ── ③ 無物體的置信度 Loss ──────────────────────
        conf_noobj_loss = self.lambda_noobj * (
            noobj_mask * pred_b1[..., 4].pow(2) +
            noobj_mask * pred_b2[..., 4].pow(2)
        ).sum()

        # ── ④ 類別 Loss ──────────────────────────
        cls_loss = (
            obj_mask.unsqueeze(-1) * (pred_cls - tgt_cls).pow(2)
        ).sum()

        # ── 總 Loss ──────────────────────────────
        total_loss = (coord_loss + conf_obj_loss + conf_noobj_loss + cls_loss) / batch_size

        return total_loss, {
            'coord':    coord_loss.item() / batch_size,
            'conf_obj': conf_obj_loss.item() / batch_size,
            'conf_noobj': conf_noobj_loss.item() / batch_size,
            'cls':      cls_loss.item() / batch_size,
        }

criterion = YOLOv1Loss(S=S, B=B, C=C)
print('Loss 函數定義完成！')
```

開始訓練

訓練時

- 最佳model會存入./checkpoints
- 輸出每種loss

```
Epoch [  1/30]  Train: 7.4401  Val: 5.2664  LR: 5.00e-04
  Loss 拆解 → coord: 4.726  conf_obj: 0.068  conf_noobj: 0.094  cls: 2.552
```

- 每隔5epoch會記錄一次

定義Decode & NMS

定義NMS (最大值抑制)

0.5

```python
def nms(boxes, iou_threshold=0.5):
    if len(boxes) == 0:
        return []

    boxes = sorted(boxes, key=lambda x:x[4], reverse=True) #按照scores去排名
    kept = []

    while boxes:
        best = boxes.pop(0) #取出最高
        kept.append(best) #直接保留

        # 把best轉換成tensor
        b1 = torch.tensor([[best[0], best[1], best[2], best[3]]])
        remaining = []
        for b in boxes:
            b2 = torch.tensor([[b[0], b[1], b[2], b[3]]])

            # 最高分的資訊
            cx1 = (b1[..., 0] + b1[..., 2]) / 2
            cy1 = (b1[..., 1] + b1[..., 3]) / 2
            w1  = b1[..., 2] - b1[..., 0]
            h1  = b1[..., 3] - b1[..., 1]
```

好

候選

```python
            # 第2、3、4個候選框框....
            cx2 = (b2[..., 0] + b2[..., 2]) / 2
            cy2 = (b2[..., 1] + b2[..., 3]) / 2
            w2  = b2[..., 2] - b2[..., 0]
            h2  = b2[..., 3] - b2[..., 1]

            iou = compute_iou(
                torch.stack([cx1, cy1, w1, h1], dim=-1), #合併成1D tensor
                torch.stack([cx2, cy2, w2, h2], dim=-1)
            ).item()

            if iou < iou_threshold:
                remaining.append(b)
        boxes = remaining

    return kept
```

Deocde預測結果

```python
def decode_prediction(pred, conf_threshold=0.3, iou_threshold=0.5, img_size=448):
    cell_size = img_size / S
    raw_boxes = []

    for row in range(S):
        for col in range(S):
            cell = pred[row, col]
            #B:bounding box數量
            # cls probs [B*5, cls0, cls1, ... ,cls19]
            cls_probs = cell[B*5:]
            cls_idx = cls_probs.argmax().item()
            cls_prob = cls_probs[cls_idx].item()

            for b in range(B):
                offset = b*5
                x_cell = cell[offset + 0].item()
                y_cell = cell[offset + 1].item()
                w_norm = cell[offset + 2].item()
                h_norm = cell[offset + 2].item()
                conf   = cell[offset + 4].item()

                score = conf * cls_prob

                if score < conf_threshold:
                    continue
```

0~1

output

分數

標準

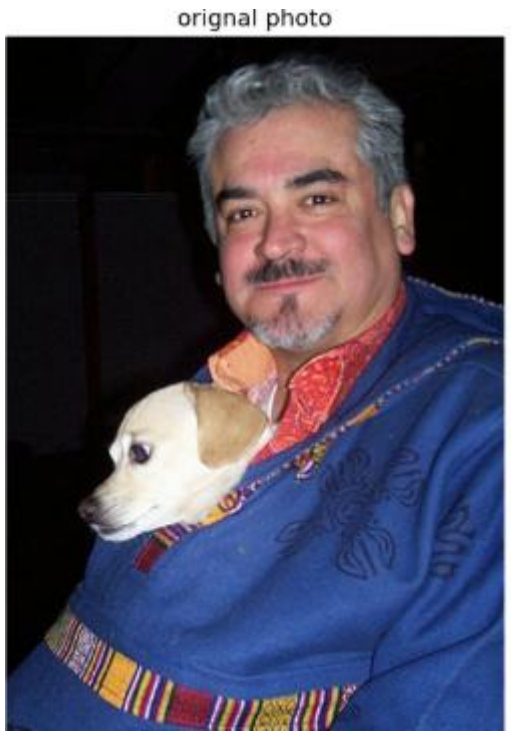
```python
                # 將標準化的座標，還原到原本規模
                cx = (col + x_cell) * cell_size
                cy = (row + y_cell) * cell_size
                w  = w_norm * img_size
                h  = h_norm * img_size

                x1 = cx - w / 2
                y1 = cy - h / 2
                x2 = cx + w / 2
                y2 = cy + h / 2

                raw_boxes.append([x1, y1, x2, y2, score, cls_idx])
    # 分類別做 NMS
    final_boxes = []
    for cls_idx in range(C):
        cls_boxes = [b for b in raw_boxes if b[5] == cls_idx]
        final_boxes.extend(nms(cls_boxes, iou_threshold))

    return final_boxes
```

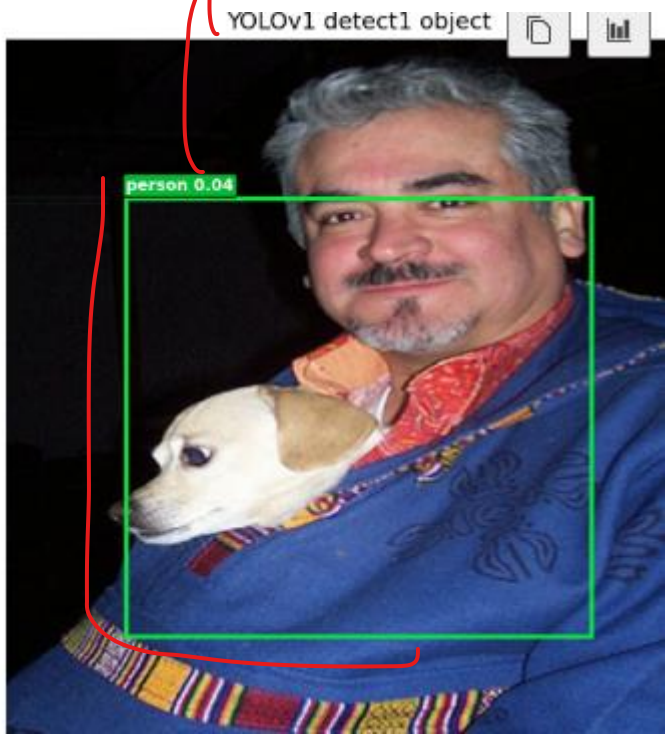
原尺度

NMS

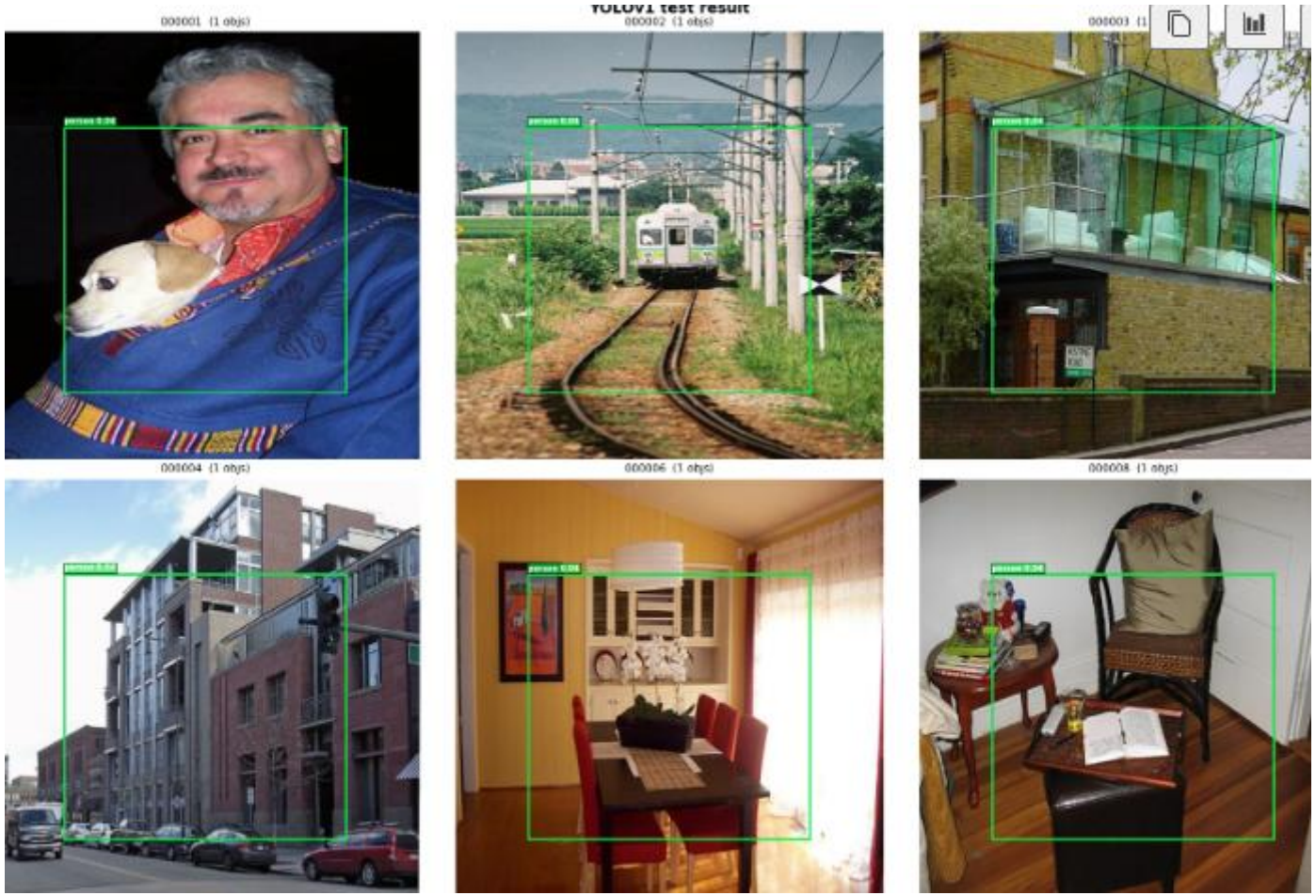
推論與視覺化

強制擬合的驗證





強制擬合的驗證



本日作業

- 作業多，開放到期中考周結束 (4/30)
- 參考簡報，完成notebook 程式碼
- 訓練YOLOv1 (若想要看到效果，建議至少訓練30 epoch)
- 進行推論程式 (若訓練不多，請把conf_threshold調低)

```
detected = inference_and_visualize(sample_img, conf_threshold=0.03)
```

請繳交程式碼、訓練日誌(有epoch、準確率的畫面)、Yolo畫的圖(有框)

一樣截圖 -> PDF

期中報告

- 做一個簡報關於期末的深度學習專題
- 下周會輪流進行口頭報告
- 報告內容須有:

主題介紹 → 選題動機 → 預計資料取得 → 使用方法 → 預計分工

參考資料

What is YOLO algorithm?
https://www.youtube.com/watch?v=ag3DLKsl2vk&t=95s

Tommy Huang-NMS:
https://chih-sheng-huang821.medium.com/機器-深度學習-物件偵測-non-maximum-suppression-nms-aa70c45adffa